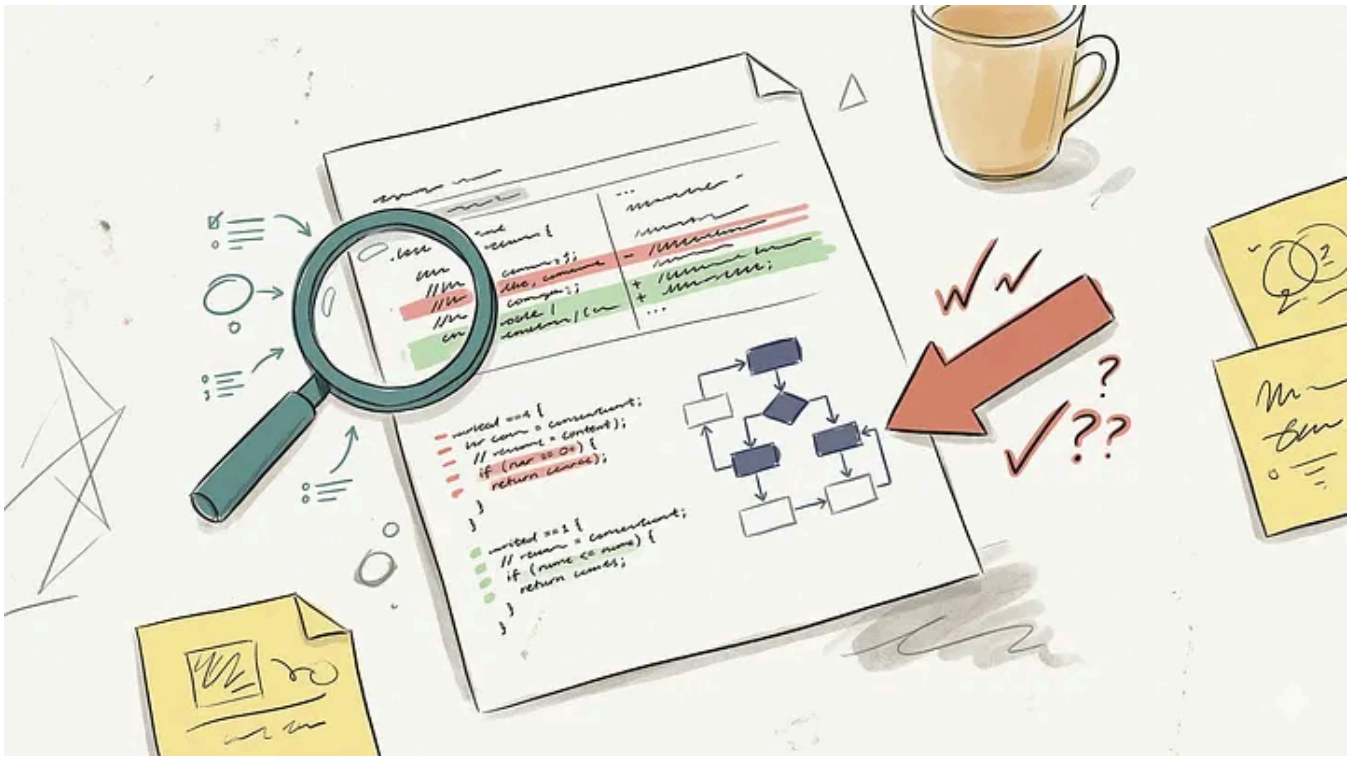
Multi-Agent Systems: When 2 Agents Beat 1

15 min read · 5 days ago

Anubhav Follow

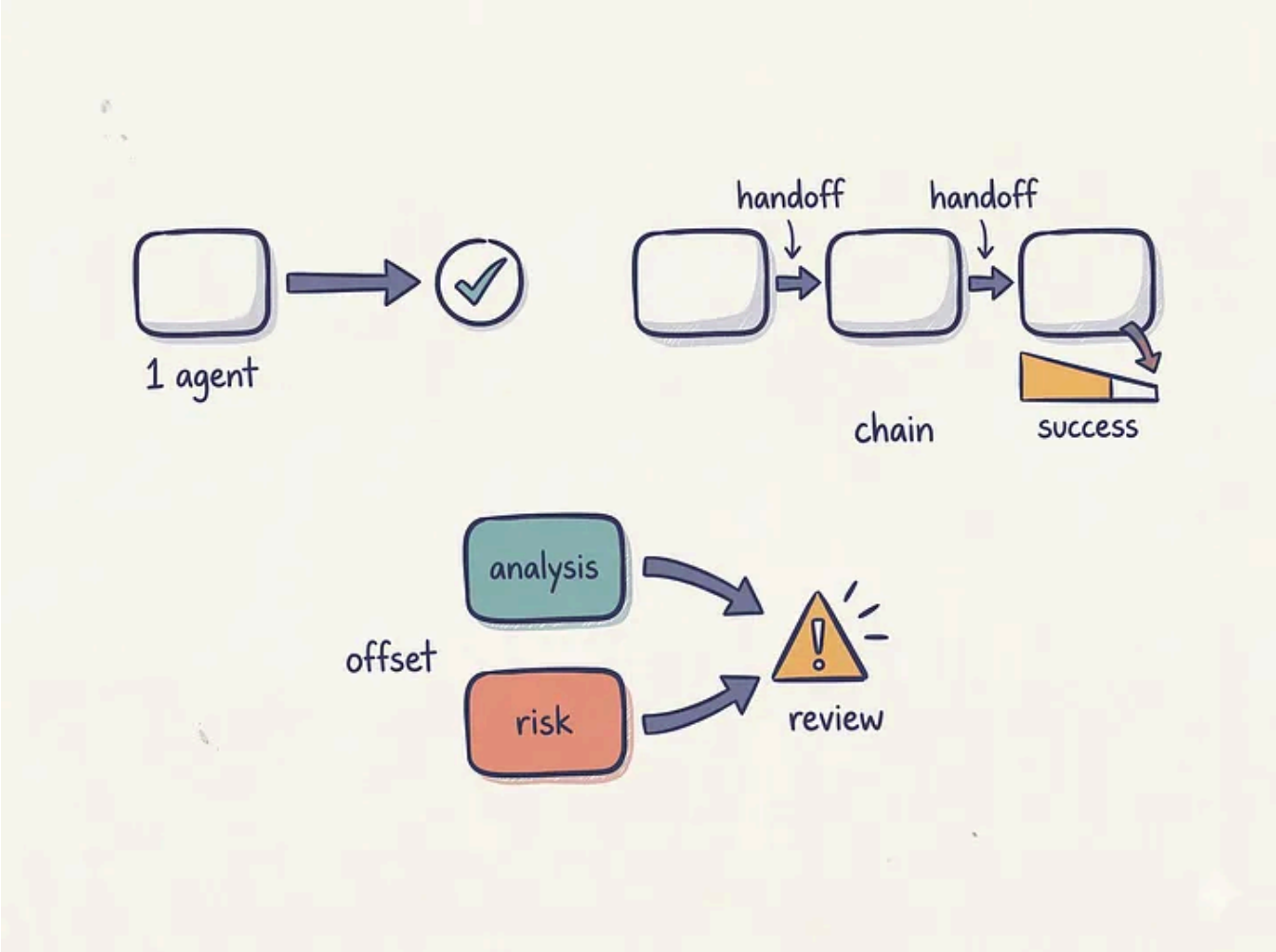
> *Read the article for free **here**.*



You see the word multi-agent everywhere right now. People build systems with five different AI personas talking to each other in a simulated chat room just to scrape a website and write a blog post. They give them names like Researcher, Writer, and Editor and watch the terminal output scroll by as the agents debate with each other. It all looks impressive but is not the right way you build software.

Adding more agents to a system does not automatically make it smarter. It actually multiplies your failure rate. Think about the basic math of probability. If you have a single agent that executes its task correctly 90% of the time, your naive system reliability is 0.90.

If you chain three of those agents together, you multiply those probabilities. Your baseline reliability just dropped to 72%. You doubled your latency, tripled your API cost, and made the final output no better.



We are going to look at exactly when it makes sense to introduce multiple agents. We will build an AI code review copilot and we will build it twice. First, we will write a standard single-agent reviewer. Then we will split the job into a two-agent architecture using an Analyzer and a Risk Reviewer. We will use modern LangGraph patterns like conditional edges, `interrupt()`, and `Command`-based resume to handle human escalation.

We will see exactly why the single agent misses a critical billing logic flaw, and why the two-agent system catches it.

The Code Review Problem

Code review is a notoriously difficult task for LLMs. A good code review requires two completely different modes of thinking.

First, you need to comprehend the change. You have to read the diff, map the modified functions, figure out which files are impacted, and understand the intent of the author. You are trying to build a mental model of the new feature.

Second, you have to look for edge cases, missing tests, and logic flaws. This is an exercise in adversarial thinking. You are trying to break the code.

When you ask an LLM to do both of these things in a single prompt, it struggles. Asking one agent to summarize and critique in one pass often biases it toward coherence over skepticism. Once it settles on a narrative of what the code does, it tends to under-invest in adversarial checking. It usually just points out minor stylistic issues like missing docstrings or variable naming conventions.

Let us look at a concrete example. We have a mock pull request for a billing system. The author is adding a discount feature to an invoice calculator.

Here is the diff we are going to feed to our agents.

```
--- a/src/billing/invoice.py
+++ b/src/billing/invoice.py
@@ -42,7 +42,9 @@
 class InvoiceService:
-    def calculate_total(self, items):
-        return sum(i.price * i.qty for i in items)
+    def calculate_total(self, items, discount_pct=0):
+        subtotal = sum(i.price * i.qty for i in items)
+        return subtotal * (1 - discount_pct)

--- a/src/billing/api.py
+++ b/src/billing/api.py
@@ -18,6 +18,8 @@
 @router.post("/invoice")
 def create_invoice(req: InvoiceRequest):
+    discount = req.discount_pct   # NEW: from user input
     svc = InvoiceService()
-    total = svc.calculate_total(req.items)
+    total = svc.calculate_total(req.items, discount)
     return {"total": total}

--- a/config/feature_flags.yaml
+++ b/config/feature_flags.yaml
@@ -5,3 +5,4 @@
 flags:
   new_dashboard: true
+  discount_billing: true   # rollout: 100% immediately
```

If you look closely at this code, you will spot a few major problems. The `discount_pct` is taken directly from user input in the API layer and passed to the

calculation service. There is no validation. A user can pass a negative discount to increase the price, or a discount greater than 1 to get a negative total. The test files are not updated to cover this new path. The feature flag is set to roll out to 100% of users immediately.

This is an abuse-prone pricing path. Let us see how a standard single-agent copilot handles it.

Design 1: The Single-Agent Reviewer

We start by defining the tools our agent can use. The agent needs to fetch the diff, read files, search for symbols, and check test coverage.

```
from langchain_core.tools import tool
import textwrap

MOCK_DIFF = """..."""  # The diff shown above
MOCK_FILES = {
    "src/billing/invoice.py": "class InvoiceService:\n    def calculate_total(s
    "src/billing/api.py": '@router.post("/invoice")\ndef create_invoice(req: Ir
    "tests/test_billing.py": "def test_calculate_total():\n    # only tests no-
}

@tool
def get_diff(pr_id: str) -> str:
    """Fetch the PR diff."""
    return MOCK_DIFF

@tool
def read_file(path: str) -> str:
    """Read a file from the repo."""
    return MOCK_FILES.get(path, f"FILE NOT FOUND: {path}")

@tool
def search_symbol(name: str) -> str:
    """Search for a symbol across the codebase."""
    if "discount" in name.lower():
        return "Found: InvoiceService.calculate_total(discount_pct) — src/billi
    return f"No results for '{name}'"

@tool
def list_tests(path: str) -> str:
    """List test files covering a source path."""
    if "billing" in path:
        return "tests/test_billing.py — covers calculate_total (no-discount pat
    return "No tests found"

ALL_TOOLS = [get_diff, read_file, search_symbol, list_tests]
```

Next we define the state schema. We use `TypedDict` for optimal compatibility with LangGraph. The state holds the PR ID, the message history, the raw diff string, and a list of findings.

We also define a Pydantic model for the findings. We force the LLM to categorize its findings using specific literals. This makes downstream processing much easier.

```
from typing import Literal
from typing_extensions import TypedDict
from pydantic import BaseModel
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import AIMessage, HumanMessage, SystemMessage
from langgraph.graph import END, START, StateGraph

class ReviewFinding(BaseModel):
    severity: Literal["critical", "high", "medium", "low", "info"]
    category: Literal["bug", "regression", "missing_test", "rollout_risk",
                      "security", "edge_case", "style"]
    file: str
    description: str
    confidence: Literal["high", "medium", "low"]

class SingleAgentState(TypedDict):
    pr_id: str
    messages: list
    diff: str
    findings: list[ReviewFinding]

llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)
```

Now we build the nodes. The graph is a simple straight line. We have a fetch node that loads the context, and a review node that runs the LLM prompt.

```
def sa_fetch(state: SingleAgentState) -> dict:
    diff = get_diff.invoke({"pr_id": state["pr_id"]})
    tests = list_tests.invoke({"path": "src/billing"})
    return {"diff": diff, "messages": [
        AIMessage(content=f"Diff loaded. Test coverage: {tests}")
    ]}

def sa_review(state: SingleAgentState) -> dict:
    """Single agent does BOTH jobs: summarize + critique in one pass."""
    prompt = f"""\
You are a senior code reviewer. Read this PR diff, summarize the change,
and produce a list of risk findings. Be specific.

DIFF:
{state['diff']}

Respond with JSON: {{"summary": "...", "findings": [
  {{"severity": "...", "category": "...", "file": "...",
    "description": "...", "confidence": "..."}}
]}}
```

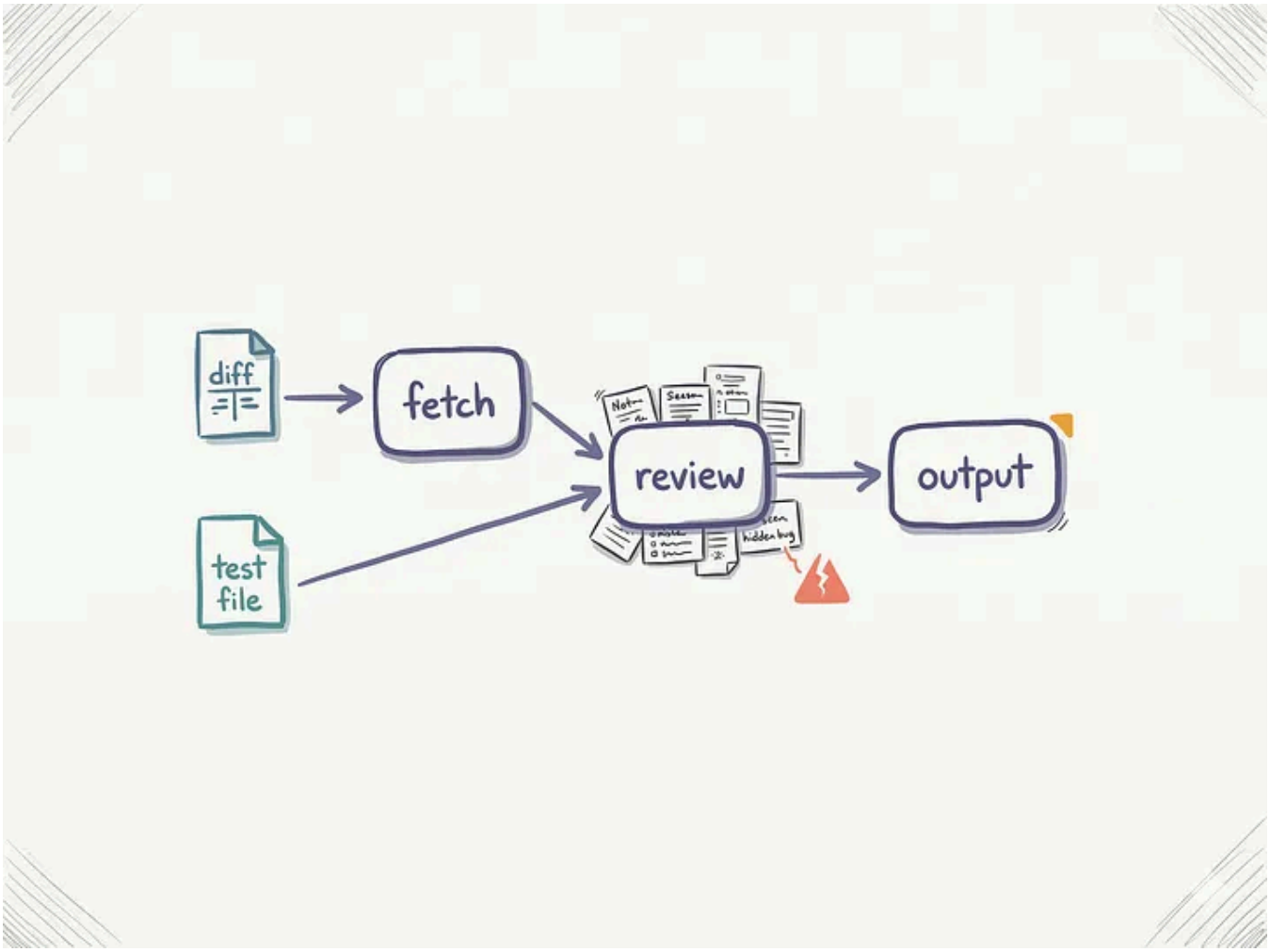
```
    """
        resp = llm.invoke([SystemMessage(content="You are a code review bot."),
                           HumanMessage(content=prompt)])

        # In a real app we parse the JSON response here.
        # We simulate the typical single-agent output for this diff.
        findings = [
            ReviewFinding(
                severity="medium",
                category="missing_test",
                file="tests/test_billing.py",
                description="Add unit tests for the new discount_pct parameter.",
                confidence="high"
            ),
            ReviewFinding(
                severity="low",
                category="style",
                file="src/billing/invoice.py",
                description="Consider adding type hints to the items list.",
                confidence="high"
            )
        ]
        return {"findings": findings, "messages": [resp]}

    def build_single_agent():
        g = StateGraph(SingleAgentState)
        g.add_node("fetch", sa_fetch)
        g.add_node("review", sa_review)
        g.add_edge(START, "fetch")
        g.add_edge("fetch", "review")
        g.add_edge("review", END)
        return g.compile()
```

This is roughly how most first-pass AI code reviewers get built. You pass the diff to the model and ask it to find bugs.

When you run this agent against our mock PR, it finds the missing test. It suggests adding type hints. It completely misses the unvalidated user input. It misses the rollout risk.

The prompt asks the model to summarize the change and produce a list of risk findings. The model spends its attention tokens explaining that a discount feature was added. By the time it starts generating the findings array, it is in a descriptive mindset. It looks at the surface level syntax. It does not actively try to break the code.

To fix this, we need to separate the synthesis task from the adversarial task. We need two agents.



The Handoff: Why Structured Schemas Matter

The biggest mistake people make with multi-agent systems is letting the agents talk to each other in raw unstructured text.

If Agent A writes a three-paragraph summary and passes it to Agent B, Agent B has to parse that text, guess the important parts, and base its work on a fuzzy foundation. This is how context degrades.

Agents are just functions. Functions should pass strictly typed objects to each other. If you want a multi-agent system to be reliable, you need to define the exact schema of the handoff artifact.

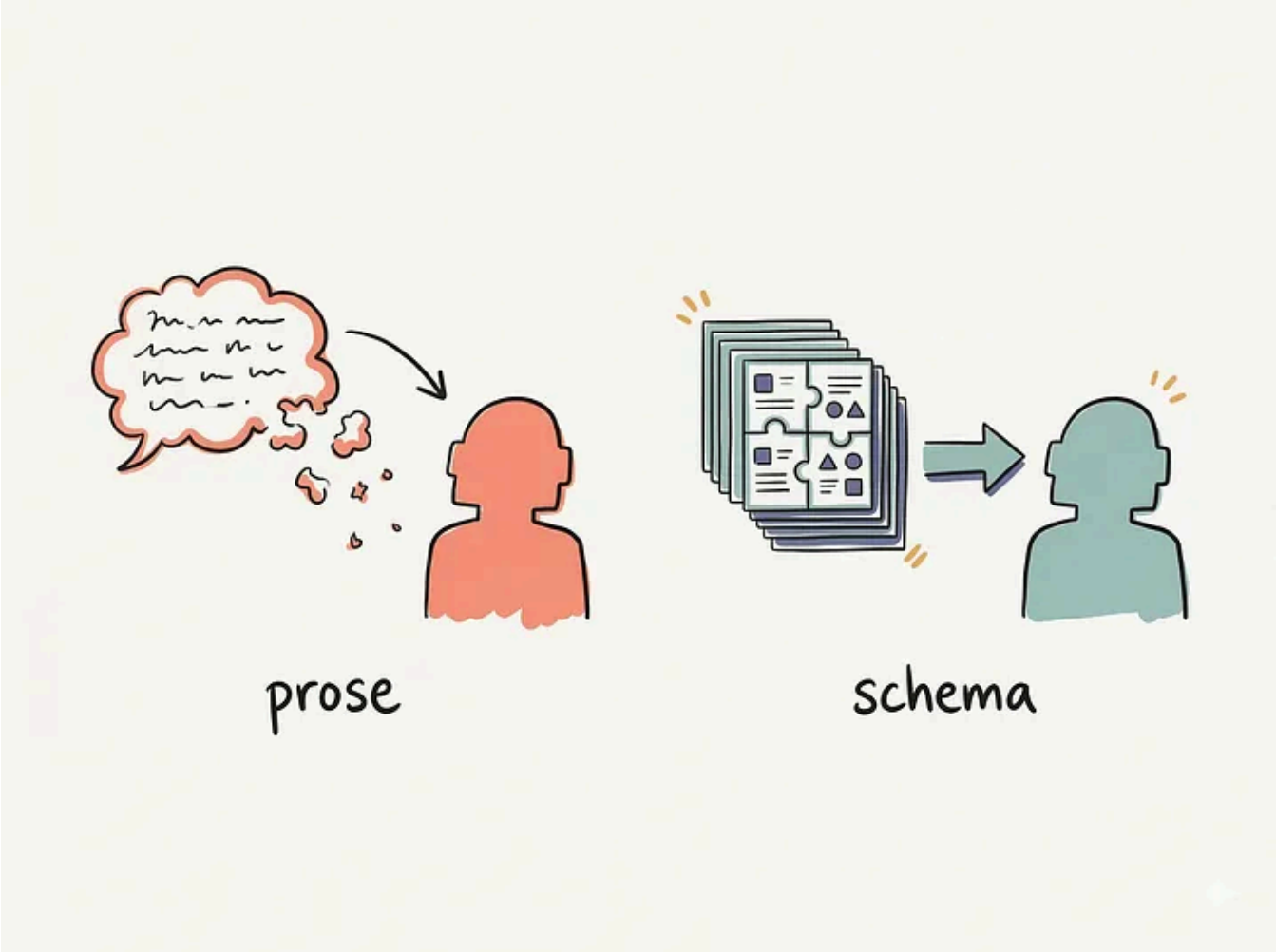
For our code review copilot, the first agent is the Analyzer. Its job is to map the PR. We force it to output a `ChangeModel`. This object forces the Analyzer to extract specific metadata about the change.

```
class ChangedInterface(BaseModel):
    file: str
    symbol: str
    change_type: Literal["added", "modified", "removed"]
    description: str

class ChangeModel(BaseModel):
    """Analyzer → Reviewer handoff. Structured, not prose."""
    purpose: str
    impacted_files: list[str]
    changed_interfaces: list[ChangedInterface]
    config_changes: list[str]
    migration_risk: bool
    assumptions: list[str]
    tests_touched: list[str]
    tests_likely_needed: list[str]
```

Every field here serves a specific purpose for the downstream agent. The `changed_interfaces` list gives the second agent a fast lookup table of what actually matters in the code. The `config_changes` list forces the Analyzer to separate application logic from deployment logic.

But look at the `assumptions` array. We force the Analyzer to explicitly list the assumptions the code author is making. The Analyzer is not critiquing the code. It is just stating facts. It looks at the code and states "The author assumes discount_pct is a fraction between 0 and 1."

This structured object becomes the exact input for the second agent.



Design 2: The Two-Agent Architecture

We define a new state schema for our two-agent graph. It holds the `ChangeModel` and a boolean flag called `conflict`. We will use this flag later to decide if we need to escalate to a human.

```
class TwoAgentState(TypedDict):
    pr_id: str
    messages: list
    diff: str
    change_model: ChangeModel | None
    findings: list[ReviewFinding]
    conflict: bool
```

The fetch node remains exactly the same. We grab the diff and the test coverage.

Now we build the Analyzer node. The prompt here is strictly constrained. We explicitly forbid the model from looking for bugs. We want it to focus entirely on comprehension.

```
ANALYZER_PROMPT = """\
You are the Analyzer agent. Your ONLY job: understand the change.
Do NOT critique. Do NOT hunt for bugs. Just map what changed.
Produce a structured change model."""

def ta_analyzer(state: TwoAgentState) -> dict:
    """Analyzer: compress & clarify. Optimizes for coherence."""

    # We use LLM structured output to fill the ChangeModel.
    # For this demonstration, we simulate the accurate analysis.
    cm = ChangeModel(
        purpose="Add discount percentage support to invoice billing",
        impacted_files=["src/billing/invoice.py", "src/billing/api.py",
                        "config/feature_flags.yaml"],
        changed_interfaces=[
            ChangedInterface(file="src/billing/invoice.py",
                symbol="InvoiceService.calculate_total",
                change_type="modified",
                description="Added discount_pct param (default 0)"),
            ChangedInterface(file="src/billing/api.py",
                symbol="create_invoice",
                change_type="modified",
                description="Reads discount_pct from request, passes to service
        ],
        config_changes=["discount_billing flag added, 100% rollout"],
        migration_risk=False,
        assumptions=[
            "discount_pct is expected to be 0..1 (fraction, not percentage)",
            "No existing callers pass discount_pct yet",
            "Feature flag controls visibility, not the calculation",
        ],
        tests_touched=[],
        tests_likely_needed=[
            "test discount path in calculate_total",
            "test boundary: discount_pct = 0, 1, >1, <0",
            "test API validation of discount_pct input",
        ],
    )
    return {
        "change_model": cm,
        "messages": [AIMessage(content=f"Analyzer: change model built. "
                               f"{len(cm.changed_interfaces)} interfaces change
                               f"{len(cm.assumptions)} assumptions made.")],
    }
```

The Analyzer reads the diff and maps the reality of the code. It notices the new parameter and a new feature flag. It writes down the implicit assumption that the

discount should be between 0 and 1 and the feature flag controls the logic.

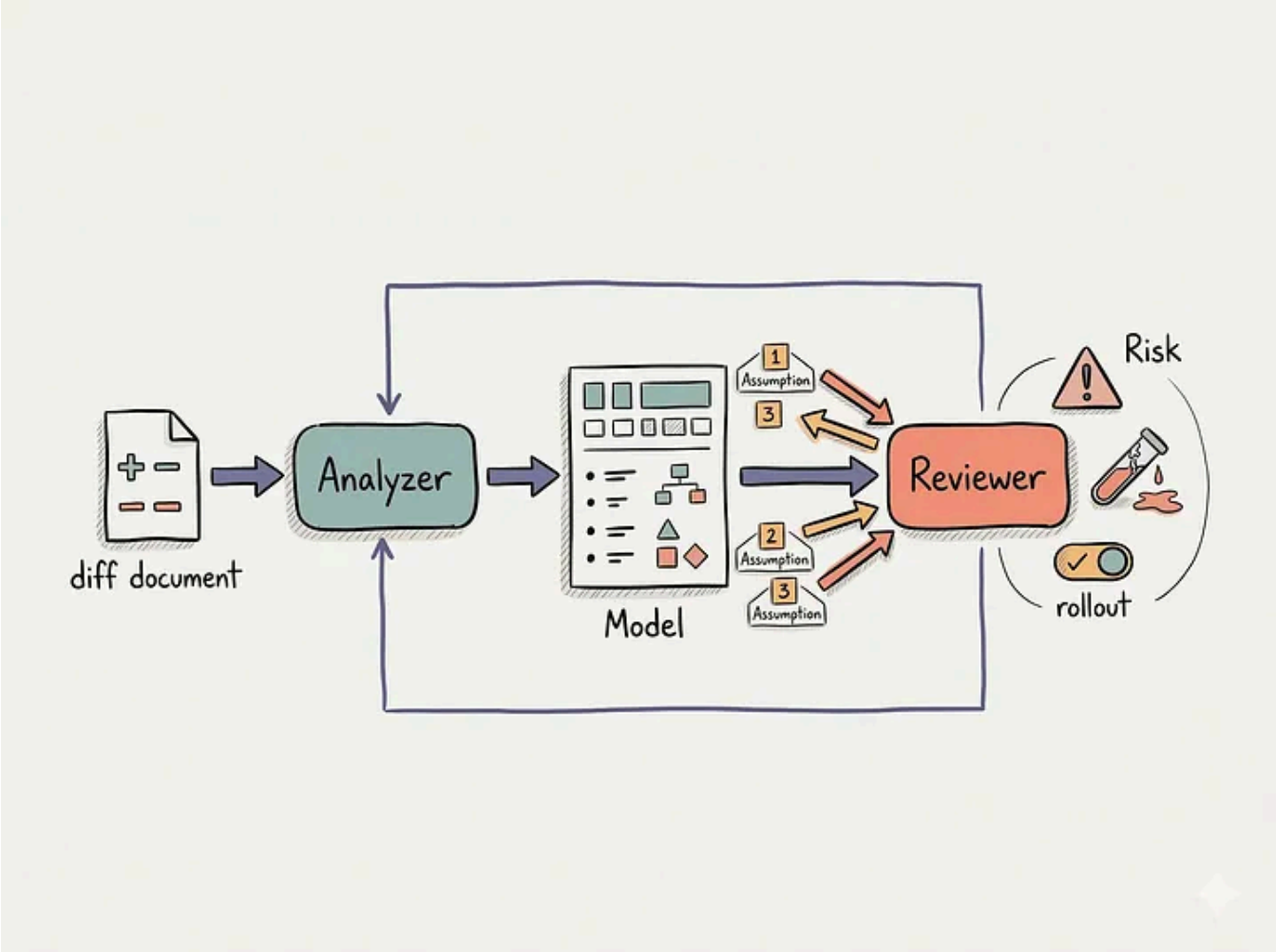
Now we hand this object over to the Risk Reviewer. The prompt for the Reviewer is highly adversarial. We tell it to distrust the Analyzer. We tell it to actively hunt for flaws.

```
REVIEWER_PROMPT = """\
You are the Risk Reviewer. The Analyzer gave you a change model.
DISTRUST it. Your job: find what's missing, broken, or dangerous.
Challenge every assumption. Check for missing tests, regressions,
rollout risks, and security issues."""

def ta_reviewer(state: TwoAgentState) -> dict:
    """Reviewer: expand & challenge. Optimizes for skepticism."""
    cm = state["change_model"]
    findings: list[ReviewFinding] = []

    # The Reviewer iterates through the Analyzer's assumptions
    for a in cm.assumptions:
        if "0..1" in a:
            findings.append(ReviewFinding(
                severity="critical",
                category="security",
                file="src/billing/api.py",
                description="ASSUMPTION CHALLENGED: discount_pct comes from use
                    "(req.discount_pct) with NO validation. Values <0 or >1 bre
                    "Negative discount = price increase beyond subtotal. "
                    "Value >1 = negative total.",
                confidence="high"))

    # The Reviewer checks the test mapping
    if not cm.tests_touched and cm.tests_likely_needed:
        findings.append(ReviewFinding(
            severity="high",
            category="missing_test",
            file="tests/test_billing.py",
            description=f"NO tests touched but {len(cm.tests_likely_needed)} ne
                + "; ".join(cm.tests_likely_needed),
            confidence="high"))

    # The Reviewer checks the config changes
    for cc in cm.config_changes:
        if "100%" in cc:
            findings.append(ReviewFinding(
                severity="high",
                category="rollout_risk",
                file="config/feature_flags.yaml",
                description="Feature flag at 100% from day one — no gradual rol
                    "Combined with unvalidated discount input, this is a billin
```

```
                confidence="high"))

    # The Reviewer looks for edge cases in the logic flow
    findings.append(ReviewFinding(
        severity="medium",
        category="edge_case",
        file="src/billing/api.py",
        description="Feature flag controls visibility but calculate_total alway
            "discount. If flag is off but API still receives discount_pct, disc
            "is silently applied.",
        confidence="medium"))

    # We determine if there is a conflict worth escalating
    conflict = any(f.severity in ("critical", "high") for f in findings)

    return {
        "findings": findings,
        "conflict": conflict,
        "messages": [AIMessage(content=f"Reviewer: {len(findings)} findings, "
                               f"conflict={conflict}")],
    }
```

Because the Reviewer does not have to spend tokens figuring out what files changed or what the purpose of the PR is, it can dedicate its entire context window to breaking the logic. It reads the assumption “discount_pct is expected to be 0..1”. It looks at the diff and it sees no validation code. It flags this as a high-severity input validation bug.

This is one of the few cases where multi-agent design earns its keep. You use one agent to build the target, and the second agent to shoot at it.



Arbitration and Human-in-the-Loop

We have a list of severe findings. The Reviewer found critical bugs that the initial PR author clearly missed.

We do not want to just post these directly to GitHub. If the Reviewer is hallucinating, it will annoy the engineering team. We can add a quick production extension here: a merge node to evaluate the findings and decide if we need a human to arbitrate.

```
def ta_merge(state: TwoAgentState) -> dict:
    """Merge findings. If conflict, surface for human review."""
    if state["conflict"]:
        return {
            "messages": [AIMessage(content=(
                "⚠ CONFLICT: Reviewer found critical/high issues. "
                "Routing to human review."
            ))],
        }
    return {
        "messages": [AIMessage(content="Findings merged. No escalation needed."
    }
```

```
def route_after_merge(state: TwoAgentState) -> str:
    return "escalate" if state["conflict"] else "emit"
```

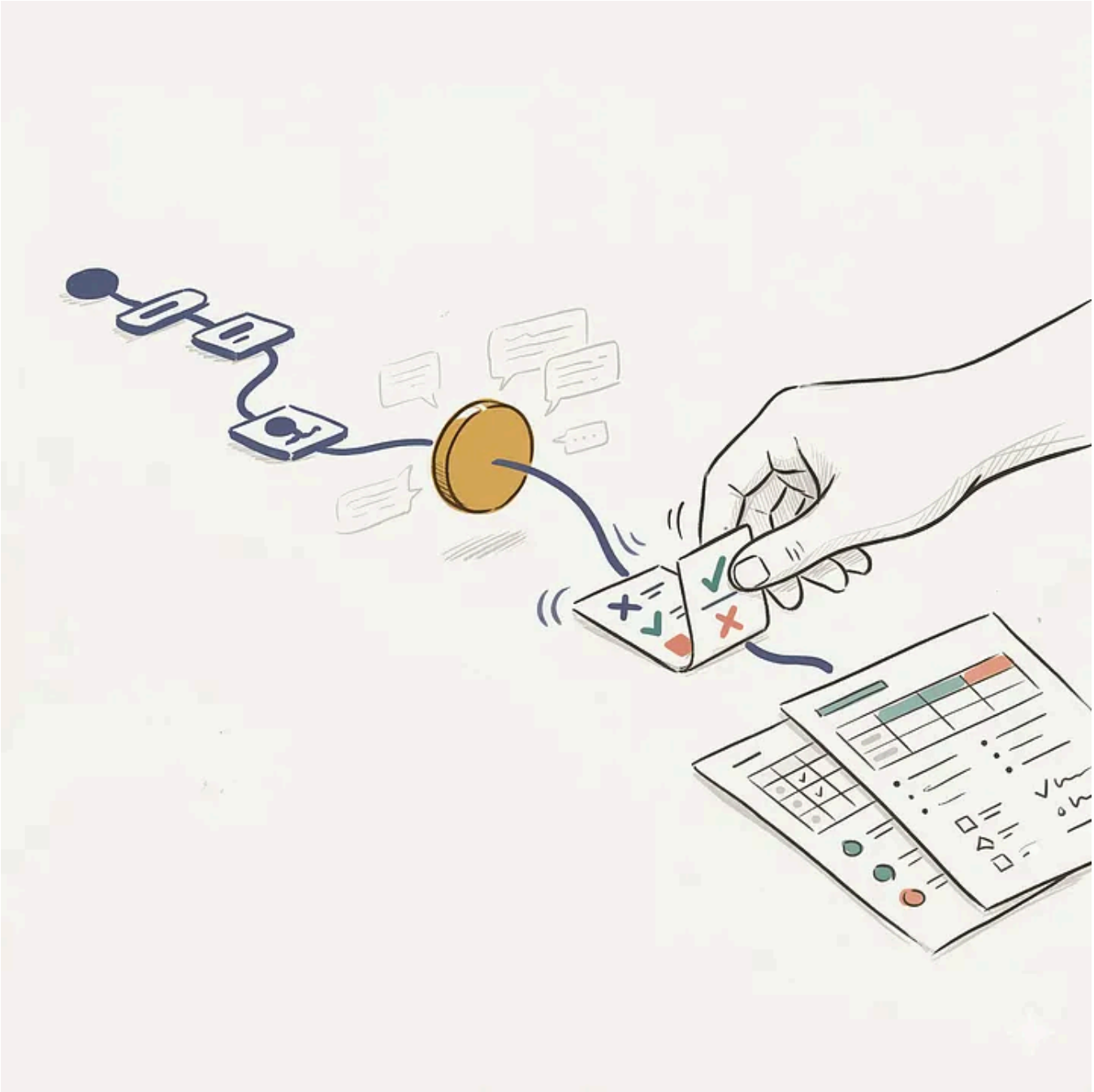
If the `conflict` flag is true, we route to the escalate node. This is where we use modern LangGraph patterns like conditional edges, `interrupt()`, and `Command`-based resume.

```
from langgraph.types import Command, interrupt

def ta_escalate(state: TwoAgentState) -> Command:
    """Human-in-the-loop for conflicting findings."""

    # The interrupt function pauses execution and surfaces data to the caller
    decision = interrupt({
        "kind": "review_conflict",
        "pr_id": state["pr_id"],
        "change_model": state["change_model"].model_dump(),
        "findings": [f.model_dump() for f in state["findings"]],
        "prompt": "Review findings. Respond: "
                  '{"action":"accept_all"} or {"action":"override","drop_indice
    })

    # Execution resumes here when the human provides input
    action = decision.get("action", "accept_all")

    if action == "override":
        drop = set(decision.get("drop_indices", []))
        kept = [f for i, f in enumerate(state["findings"]) if i not in drop]

        # We use Command to update state and dynamically route to the next node
        return Command(update={"findings": kept}, goto="emit")

    return Command(goto="emit")
```

When the graph hits the `interrupt()` call, execution pauses. It serializes the current state and saves it using the configured checkpointer. For a local demo, we use `MemorySaver`. In production, you would use a durable saver backed by Postgres so the paused state survives server restarts.

Your backend server can then send a Slack message to a senior engineer containing the structured findings. They read the critical warning about the unvalidated

discount. If they agree with the agent, they can click an “Accept All” button in Slack.



Your server receives the webhook and resumes the graph by passing a `Command` object with the resume payload.

```
# How you resume the graph from your backend API
ta.invoke(Command(resume={"action": "accept_all"}), config)
```

The graph wakes back up, processes the human decision, updates the findings if necessary, and routes to the final emit node.

```
def ta_emit(state: TwoAgentState) -> dict:
    """Final output: formatted review."""
    lines = [f"=== Code Review: PR {state['pr_id']} ==="]
    if state["change_model"]:
        lines.append(f"Purpose: {state['change_model'].purpose}")
    lines.append(f"Findings ({len(state['findings'])}):")

    for i, f in enumerate(state["findings"]):
        lines.append(f"  [{f.severity}] ({f.category}) {f.file}")
        lines.append(f"    {f.description}")

    return {"messages": [AIMessage(content="\n".join(lines))]}
```

Wiring the Two-Agent Graph

We assemble the nodes into our `StateGraph`. Notice that we do not need a static edge leaving the `escalate` node. Because `ta_escalate` returns a `Command(goto="emit")`, LangGraph handles the routing dynamically.

```
from langgraph.checkpoint.memory import MemorySaver

def build_two_agent():
    g = StateGraph(TwoAgentState)

    g.add_node("fetch", ta_fetch)
    g.add_node("analyzer", ta_analyzer)
    g.add_node("reviewer", ta_reviewer)
    g.add_node("merge", ta_merge)
    g.add_node("escalate", ta_escalate)
    g.add_node("emit", ta_emit)

    g.add_edge(START, "fetch")
    g.add_edge("fetch", "analyzer")
    g.add_edge("analyzer", "reviewer")
    g.add_edge("reviewer", "merge")

    g.add_conditional_edges("merge", route_after_merge,
        {"escalate": "escalate", "emit": "emit"})

    g.add_edge("emit", END)

    # A checkpointer is required to use interrupt()
    # Use MemorySaver for local testing, Postgres for production
    return g.compile(checkpointer=MemorySaver())
```

Running the Comparison

When you run these two architectures side by side, on this PR, the difference is hard to ignore.

The single agent finds two issues. It tells you to add unit tests and suggests some type hints. It acts like a junior developer reviewing a pull request. It is polite and focuses on syntax.

The two-agent system finds four issues. It flags the unvalidated user input as a critical business logic flaw. It points out that negative discounts will result in price increases. It flags the 100% rollout feature flag as a billing incident risk. It identifies an edge case where the discount is silently applied even if the feature flag is off.

Now, multi-agent does not guarantee better reviews every single time. It helps here because the decomposition is unusually clean. The second agent added massive signal. It did not just add noise. It caught the exact business logic flaws that take down production systems.

It worked because we split the cognitive load. The Analyzer did the heavy lifting of reading the diff and mapping the interfaces. The Reviewer received a clean, structured `ChangeModel` and focused entirely on skepticism.

The Decision Matrix: When NOT to Use Multi-Agent

This dual-agent setup is powerful. You will probably want to rewrite all your single agents into multi-agent workflows after seeing this. Resist that urge.

Multi-agent systems add latency. The two-agent code reviewer takes twice as long to run because it makes sequential LLM calls. It costs twice as much in API tokens. You have to maintain a more complex state schema and deal with handoff logic.

You should only use a multi-agent architecture if your use case fits one of these three rules.

First, use it for adversarial tasks. If you need a system to generate content and critique content, split it. Generator-Discriminator patterns are the most proven multi-agent design. One agent writes the code, the second agent writes the tests to break it. One agent drafts the email, the second agent checks it for compliance violations.

Second, use it for strict tool isolation. If you have an agent that searches the public web, and an agent that executes database queries, you do not want them sharing a brain. You want a Researcher agent that gathers public data and passes a sanitized summary to an Executor agent. This creates a hard security boundary. If the public web content contains prompt injection, it only compromises the Researcher. The Executor is isolated.

Third, use it for asymmetric context scaling. Sometimes you need to process a massive amount of data to make a very small decision. You can use a fleet of cheap, fast worker agents to read 100 different log files in parallel. They each extract the relevant error lines and pass a tiny summary to a single, expensive supervisor agent. The supervisor makes the final decision.

If your task does not fit one of these three rules, stick to a single agent. A single agent with a well-designed prompt, strict tool constraints, and a clean state schema will outperform a messy multi-agent system every time.

What's Next

We have built a system where two agents collaborate to find complex bugs. We used a structured handoff artifact to keep them aligned, and we used LangGraph's interrupt feature to bring a human into the loop when they found something dangerous.

But code review is just one pattern. What happens when the agents actually disagree with each other? What happens when the Analyzer says the code is safe, the Reviewer says it is broken, and they need to debate it to reach a consensus?

In the next post, we will dive into the Planner-Critic-Executor pattern. We will look at arbitration loops, how to prevent agents from getting stuck in endless arguments, and how to handle the specific failure modes of multi-agent negotiation.

Continue Reading

> ***LangGraph vs CrewAI vs AutoGen (2026)*** *— Compare top frameworks for routing and managing multi-agent architectures.*
>
> ***ReAct Agents in 2026:*** *— Learn to build reliable react agents using LangGraph state machines.*

> ***AI Agents Explained (2026)****— Master the foundational concepts behind modern autonomous AI agent systems.*
>
> ***I Built an AI Agent in Pure Python. Here's What I Learned****— Understand agent mechanics by building one without any heavy frameworks.*

AI Agent | AI | Machine Learning | Deep Learning | Python

Follow

Published in Data Science Collective

921K followers · Last published 1 day ago

Advice, insights, and ideas from the Medium data science community

Follow

Written by Anubhav

689 followers · 28 following

I write deep-dives on agents, RAG, evals, and what actually breaks when you ship AI systems

No responses yet

Datachamp

What are your thoughts?